



Universidade Federal do Tocantins (UFT)

Disciplina: Teoria da Computação (TC2026.2)

Atividade: Avaliativa 2 - Operações Lógicas com Máquina de Turing Multifita

Professor: Alexandre Tadeu Rossini da Silva

Alunos: Lucas Carvalho da Luz Moura

Matricula: 2020111816

Parte 1 - Projeto Formal da Máquina de Turing

1. Descrição do Algoritmo

A Máquina de Turing determinística de duas fitas opera utilizando um método de "marcação e retorno" para processar os bits correspondentes de w_1 e w_2 . A fita 1 atua como memória de trabalho (marcando bits visitados com x e y) e a fita 2 avança de forma independente apenas para registrar a saída da operação lógica.

2. Definição Formal dos Alfabetos

- **Alfabeto de Entrada (Σ):** $\{A, O, X, 0, 1, \#\}$
- **Alfabeto da Fita 1 (Γ_1):** $\{A, O, X, 0, 1, \#, x, y, B\}$
- **Alfabeto da Fita 2 (Γ_2):** $\{0, 1, B\}$

3. Conjunto de Estados (Q)

- q_0 : Estado inicial. Lê a operação e ramifica para o fluxo correto.
- q_op_find : Busca o próximo bit não marcado em w_1 . O prefixo op indica a operação (and, or, xor).
- $q_op_0_w1 / q_op_1_w1$: Memorizou o bit de w_1 (0 ou 1) e avança até encontrar o separador $\#$.

- $q_op_0_w2$ / $q_op_1_w2$: Avança pelos bits marcados de w_2 até achar o bit alvo, realiza a operação lógica pelas transições, escreve na fita 2 e inicia o rebobinamento.
- q_op_rewind : Move a cabeça da fita 1 para a esquerda até reencontrar o símbolo da operação (A , O ou X).
- q_accept : Estado final de aceitação.

4. Lógica de Identificação da Operação

A máquina bifurca a execução logo no primeiro passo ao ler o símbolo inicial (A , O ou X), garantindo que a escolha da operação dependa unicamente das regras de transição (transitando para q_and_find , q_or_find ou q_xor_find).

5. Localização e Processamento dos Bits de w_1 e w_2

Para localizar os bits correspondentes de ambas as palavras e processá-los, a máquina utiliza uma estratégia de marcação e rebobinamento passo a passo:

- **Busca em w_1 :** A cabeça de leitura avança na fita 1 até encontrar o primeiro bit não marcado. Ao lê-lo, marca-o (x para 0, y para 1) e memoriza o bit no estado atual (ex: $q_and_0_w1$).
- **Busca em w_2 :** A fita 1 avança ignorando o resto de w_1 , passa pelo separador $\#$ e ignora as marcações anteriores em w_2 . Ao encontrar o primeiro bit não marcado de w_2 , ela o utiliza para o cálculo.
- **Processamento e Escrita:** Com o bit de w_1 memorizado no estado e o bit atual lido de w_2 , a operação é efetuada através das transições. O bit em w_2 é marcado, o resultado da operação (0 ou 1) é escrito na fita 2 e a cabeça da fita 2 avança.
- **Rebobinamento:** A cabeça da fita 1 move-se totalmente para a esquerda até encontrar novamente a letra da operação, reiniciando o ciclo até encontrar o separador $\#$ imediatamente após a letra (o que engatilha a aceitação).

6. Função de Transição (δ)

Abaixo a tabela completa contendo o fluxo inicial e a lógica para as três operações (AND, OR e XOR). O padrão de movimentação é semelhante, alterando apenas a matriz de resultados lógicos ao escrever na Fita 2.

Estado Atual q	Leitura		Novo Estado p	Escrita		Movimento	
	Fita 1	Fita 2		Fita 1	Fita 2	Fita 1	Fita 2
q_0	A	B	q_and_find	A	B	R	S
q_0	O	B	q_or_find	O	B	R	S
q_0	X	B	q_xor_find	X	B	R	S

Continua na próxima página...

Tabela 1 – continuação da página anterior

Estado Atual	Leitura		Novo Estado	Escrita		Movimento	
	Fita 1	Fita 2		Fita 1	Fita 2	Fita 1	Fita 2
<i>q</i>			<i>p</i>				
Fluxo para Operação AND							
<i>q_and_find</i>	0	B	<i>q_and_0_w1</i>	x	B	R	S
<i>q_and_find</i>	1	B	<i>q_and_1_w1</i>	y	B	R	S
<i>q_and_find</i>	x	B	<i>q_and_find</i>	x	B	R	S
<i>q_and_find</i>	y	B	<i>q_and_find</i>	y	B	R	S
<i>q_and_find</i>	#	B	<i>q_accept</i>	#	B	S	S
<i>q_and_0_w1</i>	0,1	B	<i>q_and_0_w1</i>	0,1	B	R	S
<i>q_and_1_w1</i>	0,1	B	<i>q_and_1_w1</i>	0,1	B	R	S
<i>q_and_0_w1</i>	#	B	<i>q_and_0_w2</i>	#	B	R	S
<i>q_and_1_w1</i>	#	B	<i>q_and_1_w2</i>	#	B	R	S
<i>q_and_0_w2</i>	x,y	B	<i>q_and_0_w2</i>	x,y	B	R	S
<i>q_and_1_w2</i>	x,y	B	<i>q_and_1_w2</i>	x,y	B	R	S
<i>q_and_0_w2</i>	0	B	<i>q_and_rewind</i>	x	0	L	R
<i>q_and_0_w2</i>	1	B	<i>q_and_rewind</i>	y	0	L	R
<i>q_and_1_w2</i>	0	B	<i>q_and_rewind</i>	x	0	L	R
<i>q_and_1_w2</i>	1	B	<i>q_and_rewind</i>	y	1	L	R
<i>q_and_rewind</i>	0,1,#,x,y	B	<i>q_and_rewind</i>	0,1,#,x,y	B	L	S
<i>q_and_rewind</i>	A	B	<i>q_and_find</i>	A	B	R	S
Fluxo para Operação OR							
<i>q_or_find</i>	0	B	<i>q_or_0_w1</i>	x	B	R	S
<i>q_or_find</i>	1	B	<i>q_or_1_w1</i>	y	B	R	S
<i>q_or_find</i>	x	B	<i>q_or_find</i>	x	B	R	S
<i>q_or_find</i>	y	B	<i>q_or_find</i>	y	B	R	S
<i>q_or_find</i>	#	B	<i>q_accept</i>	#	B	S	S
<i>q_or_0_w1</i>	0,1	B	<i>q_or_0_w1</i>	0,1	B	R	S
<i>q_or_1_w1</i>	0,1	B	<i>q_or_1_w1</i>	0,1	B	R	S
<i>q_or_0_w1</i>	#	B	<i>q_or_0_w2</i>	#	B	R	S
<i>q_or_1_w1</i>	#	B	<i>q_or_1_w2</i>	#	B	R	S
<i>q_or_0_w2</i>	x,y	B	<i>q_or_0_w2</i>	x,y	B	R	S
<i>q_or_1_w2</i>	x,y	B	<i>q_or_1_w2</i>	x,y	B	R	S
<i>q_or_0_w2</i>	0	B	<i>q_or_rewind</i>	x	0	L	R
<i>q_or_0_w2</i>	1	B	<i>q_or_rewind</i>	y	1	L	R
<i>q_or_1_w2</i>	0	B	<i>q_or_rewind</i>	x	1	L	R
<i>q_or_1_w2</i>	1	B	<i>q_or_rewind</i>	y	1	L	R
<i>q_or_rewind</i>	0,1,#,x,y	B	<i>q_or_rewind</i>	0,1,#,x,y	B	L	S
<i>q_or_rewind</i>	O	B	<i>q_or_find</i>	O	B	R	S
Fluxo para Operação XOR							
<i>q_xor_find</i>	0	B	<i>q_xor_0_w1</i>	x	B	R	S
<i>q_xor_find</i>	1	B	<i>q_xor_1_w1</i>	y	B	R	S
<i>q_xor_find</i>	x	B	<i>q_xor_find</i>	x	B	R	S
<i>q_xor_find</i>	y	B	<i>q_xor_find</i>	y	B	R	S

Continua na próxima página...

Tabela 1 – continuação da página anterior

Estado Atual <i>q</i>	Leitura		Novo Estado <i>p</i>	Escrita		Movimento	
	Fita 1	Fita 2		Fita 1	Fita 2	Fita 1	Fita 2
<i>q_xor_find</i>	#	B	<i>q_accept</i>	#	B	S	S
<i>q_xor_0_w1</i>	0,1	B	<i>q_xor_0_w1</i>	0,1	B	R	S
<i>q_xor_1_w1</i>	0,1	B	<i>q_xor_1_w1</i>	0,1	B	R	S
<i>q_xor_0_w1</i>	#	B	<i>q_xor_0_w2</i>	#	B	R	S
<i>q_xor_1_w1</i>	#	B	<i>q_xor_1_w2</i>	#	B	R	S
<i>q_xor_0_w2</i>	x,y	B	<i>q_xor_0_w2</i>	x,y	B	R	S
<i>q_xor_1_w2</i>	x,y	B	<i>q_xor_1_w2</i>	x,y	B	R	S
<i>q_xor_0_w2</i>	0	B	<i>q_xor_rewind</i>	x	0	L	R
<i>q_xor_0_w2</i>	1	B	<i>q_xor_rewind</i>	y	1	L	R
<i>q_xor_1_w2</i>	0	B	<i>q_xor_rewind</i>	x	1	L	R
<i>q_xor_1_w2</i>	1	B	<i>q_xor_rewind</i>	y	0	L	R
<i>q_xor_rewind</i>	0,1,#,x,y	B	<i>q_xor_rewind</i>	0,1,#,x,y	B	L	S
<i>q_xor_rewind</i>	X	B	<i>q_xor_find</i>	X	B	R	S

7. Diagrama de Transição da Máquina de Turing

A imagem a seguir apresenta o grafo desenhado que ilustra os estados e as transições formalizadas na tabela acima.

Parte 2 - Implementação em Python

Nota técnica: A simulação não utiliza operadores nativos de lógica bit a bit do Python, baseando-se estritamente nas transições de estados definidas na Parte 1.

```
1 import time
2
3 class MaquinaTuring:
4     def __init__(self, entrada):
5         # A fita 1 permite marcações
6         self.fita1 = list(entrada)
7         # A fita 2 começa em branco e não é memória auxiliar
8         self.fita2 = ["B"]
9         self.cabeca1 = 0
10        self.cabeca2 = 0
11        self.estado = "q0"
12        self.transicoes = self._gerar_transicoes()
13
14    def _gerar_transicoes(self):
15        """Gera o dicionário de transições para as 3 operações
16        simulando uma MT."""
17        t = {}
18        # Estado Inicial
19        t[("q0", "A", "B")] = ("q_and_find", "A", "B", "R", "S")
20        t[("q0", "0", "B")] = ("q_or_find", "0", "B", "R", "S")
21        t[("q0", "X", "B")] = ("q_xor_find", "X", "B", "R", "S")
22
23        operacoes = {
24            "and": {"simbolo": "A", "logica": {(0,0):0, (0,1):0, (1,0):0, (1,1):1}},
25            "or": {"simbolo": "0", "logica": {(0,0):0, (0,1):1, (1,0):1, (1,1):1}},
26            "xor": {"simbolo": "X", "logica": {(0,0):0, (0,1):1, (1,0):1, (1,1):0}}
27        }
28
29        for op, dados in operacoes.items():
30            simbolo = dados["simbolo"]
31            logica = dados["logica"]
32
33            # Buscar bit não marcado em w1
34            t[(f"q_{op}_find", "x", "B")] = (f"q_{op}_find", "x", "B", "R", "S")
35            t[(f"q_{op}_find", "y", "B")] = (f"q_{op}_find", "y", "B", "R", "S")
36            t[(f"q_{op}_find", "0", "B")] = (f"q_{op}_0_w1", "x", "B", "R", "S")
37            t[(f"q_{op}_find", "1", "B")] = (f"q_{op}_1_w1", "y", "B", "R", "S")
38            t[(f"q_{op}_find", "#", "B")] = ("q_accept", "#", "B", "S", "S")
39
40            # Avançar w1 até encontrar o '#'
41            for val in ["0", "1"]:
42                t[(f"q_{op}_0_w1", val, "B")] = (f"q_{op}_0_w1", val, "B", "R", "S")
43                t[(f"q_{op}_1_w1", val, "B")] = (f"q_{op}_1_w1", val, "B", "R", "S")
```

```

43         t[(f"q_{op}_0_w1", "#", "B")] = (f"q_{op}_0_w2", "#", "B", "
R", "S")
44         t[(f"q_{op}_1_w1", "#", "B")] = (f"q_{op}_1_w2", "#", "B", "
R", "S")
45
46         # Avançar marcadores de w2
47         for val in ["x", "y"]:
48             t[(f"q_{op}_0_w2", val, "B")] = (f"q_{op}_0_w2", val, "B
", "R", "S")
49             t[(f"q_{op}_1_w2", val, "B")] = (f"q_{op}_1_w2", val, "B
", "R", "S")
50
51         # Processar o bit alvo de w2 e calcular resultado
52         res_00 = str(logica[(0,0)])
53         res_01 = str(logica[(0,1)])
54         res_10 = str(logica[(1,0)])
55         res_11 = str(logica[(1,1)])
56
57         t[(f"q_{op}_0_w2", "0", "B")] = (f"q_{op}_rewind", "x",
res_00, "L", "R")
58         t[(f"q_{op}_0_w2", "1", "B")] = (f"q_{op}_rewind", "y",
res_01, "L", "R")
59         t[(f"q_{op}_1_w2", "0", "B")] = (f"q_{op}_rewind", "x",
res_10, "L", "R")
60         t[(f"q_{op}_1_w2", "1", "B")] = (f"q_{op}_rewind", "y",
res_11, "L", "R")
61
62         # Rebobinar até a primeira posição (operação)
63         for val in ["0", "1", "x", "y", "#"]:
64             t[(f"q_{op}_rewind", val, "B")] = (f"q_{op}_rewind", val
, "B", "L", "S")
65             t[(f"q_{op}_rewind", simbolo, "B")] = (f"q_{op}_find",
simbolo, "B", "R", "S")
66
67         return t
68
69     def formatar_fita(self, fita, cabeca):
70         res = ""
71         for i, val in enumerate(fita):
72             if i == cabeca: res += f"[{val}]"
73             else: res += val
74         if cabeca >= len(fita): res += "[B]"
75         return res
76
77     def imprimir_estado(self):
78         """Imprime a simulação a cada passo"""
79         f1_str = self.formatar_fita(self.fita1, self.cabeca1)
80         f2_str = self.formatar_fita(self.fita2, self.cabeca2)
81         print(f"Estado: {self.estado:<15} | Fita 1: {f1_str:<25} (Pos: {
self.cabeca1}) | Fita 2: {f2_str:<15} (Pos: {self.cabeca2})")
82
83     def executar(self, exibir_passos=False):
84         if exibir_passos:
85             print("\n--- INICIANDO EXECUÇÃO ---")
86             self.imprimir_estado()
87
88         while self.estado != "q_accept":

```

```

89         s1 = self.fita1[self.cabeca1] if self.cabeca1 < len(self.
fita1) else "B"
90         s2 = self.fita2[self.cabeca2] if self.cabeca2 < len(self.
fita2) else "B"
91
92         chave = (self.estado, s1, s2)
93         if chave not in self.transicoes:
94             print(f"Erro: Transição não definida para {chave}")
95             break
96
97         n_estado, n_s1, n_s2, m1, m2 = self.transicoes[chave]
98
99         # Escrever fitas
100         if self.cabeca1 < len(self.fita1): self.fita1[self.cabeca1]
= n_s1
101         else: self.fita1.append(n_s1)
102
103         if self.cabeca2 < len(self.fita2): self.fita2[self.cabeca2]
= n_s2
104         else: self.fita2.append(n_s2)
105
106         self.estado = n_estado
107
108         # Movimentar cabeças
109         if m1 == "R": self.cabeca1 += 1
110         elif m1 == "L": self.cabeca1 = max(0, self.cabeca1 - 1)
111
112         if m2 == "R": self.cabeca2 += 1
113         elif m2 == "L": self.cabeca2 = max(0, self.cabeca2 - 1)
114
115         if exhibir_passos:
116             self.imprimir_estado()
117
118         # O resultado na fita 2 (ignora os B's sobressalentes)
119         resultado = "".join(self.fita2).replace("B", "")
120         return resultado
121
122 # ----- CASOS DE TESTE -----
123 testes = [
124     ("A0#0", "0"),
125     ("A101#110", "100"),
126     ("001#11", "11"),
127     ("0101#110", "111"),
128     ("X01#11", "10"),
129     ("X101#110", "011")
130 ]
131
132 print("="*60)
133 print("EXECUÇÃO DOS CASOS DE TESTE")
134 print("="*60)
135
136 for entrada, esperado in testes:
137     exhibir_rastreio = True if entrada == "A0#0" else False
138
139     mt = MaquinaTuring(entrada)
140     obtido = mt.executar(exibir_passos=exibir_rastreio)
141
142     status = "APROVADO" if obtido == esperado else "FALHOU"

```

```
print(f"\nResumo do Teste: {entrada} -> esperado {esperado} ->
      obtido {obtido} [{status}]")
```

Código-fonte completo disponível no GitHub: https://github.com/luc4sm0ur4/Teoria_computacao

Parte 3 - Testes

Entrada na fita 1	Resultado esperado na fita 2
A0#0	0
A101#110	100
O01#11	11
O101#110	111
X01#11	10
X101#110	011

```
Lucas Moura@lucasmoura MINGW64 ~/Desktop/Teoria_computacao/Trabalho2
$ py maquina_turing.py
=====
EXECUÇÃO DOS CASOS DE TESTE
=====

--- INICIANDO EXECUÇÃO ---
Estado: q0          | Fita 1: [A]0#0          (Pos: 0) | Fita 2: [B]              (Pos: 0)
Estado: q_and_find  | Fita 1: A[0]#0          (Pos: 1) | Fita 2: [B]              (Pos: 0)
Estado: q_and_0_w1  | Fita 1: Ax[#]0          (Pos: 2) | Fita 2: [B]              (Pos: 0)
Estado: q_and_0_w2  | Fita 1: Ax#[0]          (Pos: 3) | Fita 2: [B]              (Pos: 0)
Estado: q_and_rewind | Fita 1: Ax[#]x          (Pos: 2) | Fita 2: 0[B]             (Pos: 1)
Estado: q_and_rewind | Fita 1: A[x]#x          (Pos: 1) | Fita 2: 0[B]             (Pos: 1)
Estado: q_and_rewind | Fita 1: [A]x#x          (Pos: 0) | Fita 2: 0[B]             (Pos: 1)
Estado: q_and_find  | Fita 1: A[x]#x          (Pos: 1) | Fita 2: 0[B]             (Pos: 1)
Estado: q_and_find  | Fita 1: Ax[#]x          (Pos: 2) | Fita 2: 0[B]             (Pos: 1)
Estado: q_accept    | Fita 1: Ax[#]x          (Pos: 2) | Fita 2: 0[B]             (Pos: 1)

Resumo do Teste: A0#0 -> esperado 0 -> obtido 0 [APROVADO]

Resumo do Teste: A101#110 -> esperado 100 -> obtido 100 [APROVADO]

Resumo do Teste: O01#11 -> esperado 11 -> obtido 11 [APROVADO]

Resumo do Teste: O101#110 -> esperado 111 -> obtido 111 [APROVADO]

Resumo do Teste: X01#11 -> esperado 10 -> obtido 10 [APROVADO]

Resumo do Teste: X101#110 -> esperado 011 -> obtido 011 [APROVADO]
```